

Lappeenrannan teknillinen yliopisto
School of Engineering Science
Software Development Skills Full-Stack, Online course

LEARNING DIARY

Małgorzata Jagieła, 003003978

Learning Diary

Preparation & setup

19.05

- Initializing git and creating a new remote repository
- Installing node.js
- Installing Postman app

Crash Courses:

19.05

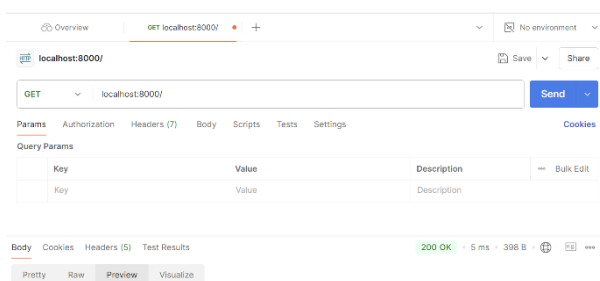
By following a video tutorial “Node.js Crash Course”:

- A Node.js project was initialized and basic dependencies were configured.
- A simple Express.js server was created to handle HTTP requests.
- Routes and a controller (postController.js) were implemented to support basic CRUD operations.
- A utility module (utils.js) was added with helper functions.
- Git was used to track changes and manage branches.

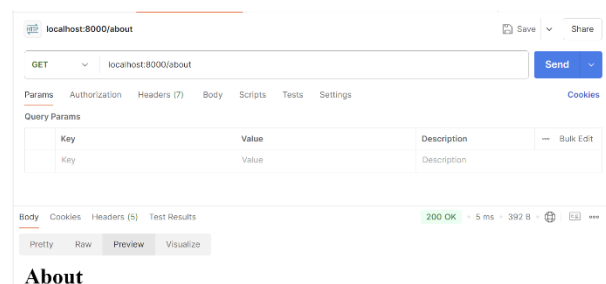
20.05

By following a video tutorial “Node.js Crash Course”:

- Added environment configuration using .env and dotenv.
- Implemented basic POST and GET routes with Express.
- Expanded the controller to handle data storage in an in-memory array.
- Refactored code to improve structure and reusability.
- Practiced using Git for branching, committing, and merging.
- Verified API functionality using Postman.

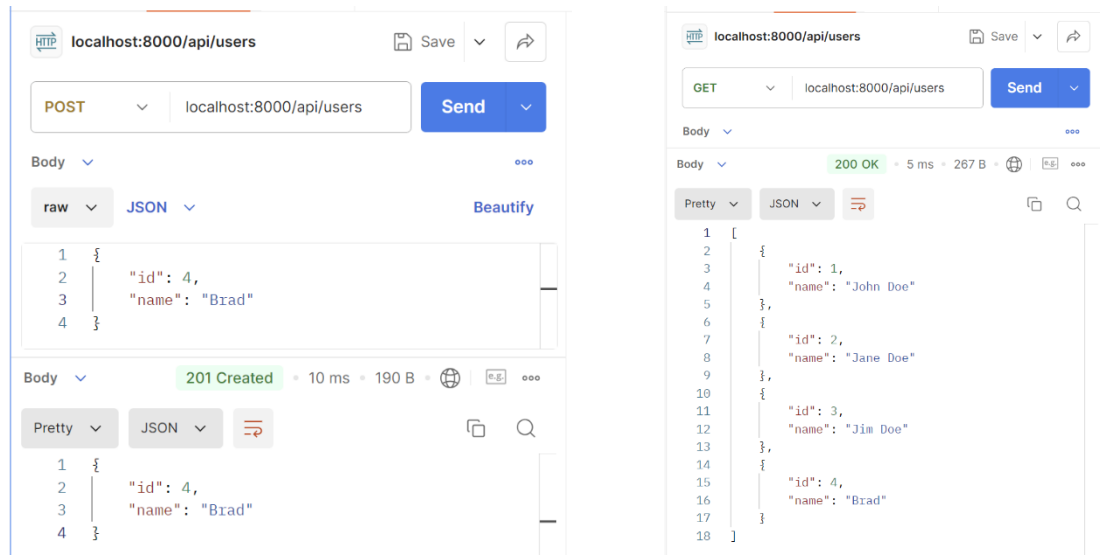


Homepage



About

- Created a new Express server with multiple test routes.
- Implemented basic GET, POST, PUT, and DELETE endpoints.
- Handled JSON data in request bodies and responses.
- Simulated a simple API using hardcoded data.
- Tested the routes using Postman.



- Tested basic methods within the: fs, os, path, crypto, events and process modules

By following a video tutorial “MongoDB Crash Course 2022”:

- Installed Mongo Shell
- Tested basic commands for managing a Mongo data base



- Connected created database to VSCode
- Created a simple aggregation

By following a video tutorial “Express Crash Course”:

- Set up Express server with ES modules
- Created in-memory data for posts
- Implemented full CRUD API (GET, POST, PUT, DELETE)
- Used Express Router for cleaner structure
- Handled query params, path params, and validation
- Added body parsing middleware for JSON

The left screenshot shows a POST request to `localhost:8000/api/posts/` with a body of `{ "id": 2, "title": "Post Two" }, { "id": 3, "title": "Post Three" }, { "id": 4, "title": "yes" }`. The response is `201 Created` with a status of 201, a time of 8 ms, and a size of 351 B. The right screenshot shows a GET request to `localhost:8000/api/posts/4` with a response of `200 OK` with a status of 200, a time of 5 ms, and a size of 257 B. The response body is `{ "id": 4, "title": "yes" }`.

- Added modular error handling with custom middleware
- Implemented `notFound` and `centralized errorHandler`
- Used `next(error)` pattern in all routes
- Confirmed 404 and validation errors return proper status and message

The left screenshot shows a GET request to `localhost:8000/api/test` with a response of `404 Not Found` with a status of 404, a time of 25 ms, and a size of 261 B. The response body is `{ "msg": "Not Found" }`. The right screenshot shows a POST request to `localhost:8000/api/posts/` with a body of `{ "id": 4, "title": "yes" }` and a response of `500 Internal Server Error` with a status of 500, a time of 6 ms, and a size of 317 B. The response body is `{ "msg": "Cannot read properties of undefined (reading 'title')" }`.

- Added a controller file to separate request handling logic from routing.
- Added a simple frontend using vanilla JavaScript
- Used `fetch()` to get and send posts to the API
- Updated the DOM dynamically based on API responses
- Handled form submission and button click events
- Set up EJS view engine in Express
- Rendered EJS view with dynamic data (title, message, people)
- Used `partials` with `include()` to reuse HTML fragments

21.05

By following a video tutorial “React JS Crash Course”:

- Initialized React project using create-react-app.
- Installed json-server for mock backend.
- Created db.json with initial task data.
- Configured json-server to run on port 5000.
- Implemented App.js with state management using useState.
- Created Tasks and Task components to display task list.
- Added functionality to delete tasks.
- Implemented toggle for task reminders.
- Integrated react-router-dom for routing.
- Created About and Footer components with navigation links.
- Added form to add new tasks.
- Handled form submission to update task list.
- Tested application functionality in the browser.

Exercises:

21.05

By following a video tutorial “Learn The MERN Stack - Express & MongoDB Rest API”:

- Initialized Node.js project with npm init.
- Installed dependencies: express, mongoose, dotenv, colors, express-async-handler.
- Created server.js and configured Express application.
- Added JSON and URL-encoded middleware.
- Added custom error-handling middleware.
- Created .env file and added environment variables.
- Configured MongoDB connection using Mongoose in db.js.
- Created Mongoose schema in goalModel.js.
- Implemented CRUD operations in goalController.js.
- Defined routes in goalRoutes.js.
- Integrated routes into the server.
- Added scripts to package.json for running server with and without nodemon.
- Fixed environment variable loading by setting explicit path in dotenv.config().
- Tested API using Postman.
- Cleaned up and refactored code.

By following a video tutorial “Learn The MERN Stack - JWT Authentication”:

- Created userController.js with functions for user registration and login.
- Defined User model schema in userModel.js using Mongoose.
- Established routes for user-related operations in userRoutes.js.
- Integrated user routes into the Express application.
- Installed and configured bcryptjs for password hashing.
- Installed and configured jsonwebtoken for token generation and verification.
- Implemented JWT-based authentication in user controller functions.
- Added middleware to protect routes requiring authentication.
- Tested user registration and login endpoints using Postman.

By following a video tutorial “Learn The MERN Stack - Frontend Authentication | Redux Toolkit”:

- Initialized React frontend within the project.
- Created components: Header, Tasks, and Task for the UI.
- Implemented state management using React's useState and useEffect hooks.
- Integrated frontend with backend API to fetch and display tasks.
- Implemented user authentication using JWT.
- Created userModel.js to define the user schema.
- Developed userController.js with registration and login functionalities.
- Added authMiddleware.js to protect routes requiring authentication.
- Updated goalRoutes.js to include authentication middleware for protected endpoints.

By following a video tutorial “Learn The MERN Stack - Redux Goals & Deploy”:

- Created Goal model with text and user fields
- Added controller for creating, fetching, and deleting user goals
- Set up API routes for goal CRUD operations
- Implemented protect middleware to secure routes
- Integrated Redux Toolkit with async thunks for goals
- Added React components for displaying and deleting goals
- Configured Express to serve built frontend in production
- Handled unknown routes by sending index.html for React Router

16.06

- Fixed errors with Registration and deleting Goals

Project:

22.05

- Initialized Node.js backend for the MyBooks project.
- Installed necessary dependencies: express, mongoose, cors, dotenv, bcryptjs, jsonwebtoken, express-async-handler.
- Created initial folder structure and server.js file.
- Connected backend to MongoDB Atlas using Mongoose and .env configuration.
- Set up basic Express server with test route.
- Configured nodemon for development.
- Verified server startup and database connection in terminal.

05.06

- Created Mongoose models for User and Book.
- Implemented JWT-based authentication: register, login, get current user.
- Created protected routes for books: get, add, mark as read, delete.
- Built userController and bookController with error handling using express-async-handler.
- Set up authMiddleware to verify tokens.
- Tested all routes using Postman.

16.06

- Fixed errors with Login and Registration
- Added booksSlice with thunks: createBook, getBooks, deleteBook.
- Created bookService for API calls with token auth.
- Managed loading, success, and error states in Redux.
- Used token from Redux state in thunks.
- Connected Redux to components and tested actions.
- Created BookItem component to display individual book entries with actions.
- Refactored Redux slice for books with async thunks and extra reducers.
- Added readBook API and thunk to mark books as read.
- Implemented conditional "I've read it!" button for unread books.
- Updated Dashboard.jsx to show separate sections for "to-read" and "read" books.

Content of the .env files

Exercises:

NODE_ENV=development

PORT=5000

MONGO_URI=mongodb+srv://gosia:gosia@exercisescluster.8w81i2j.mongodb.net/exercises?retryWrites=true&w=majority&appName=ExercisesCluster

JWT_SECRET=abc123

Project:

PORT=5000

MONGO_URI=mongodb+srv://gosia:gosia@library.5alnbwx.mongodb.net/?retryWrites=true&w=majority&appName=library

JWT_SECRET=supersecretkey